AICE3002/AICE6001 Laboratory Portfolio
Investigation 4 – Transforming Sequences

- .-. .- -. ... ..-. — .-. – .. -. –. / ... . –.- ..- . -. -.-. . ...

Jonathon Hare & Antonia Marcu
{j.s.hare, a.marcu}@soton.ac.uk

2026–27

This is the final individual investigation in the laboratory portfolio. It should be completed after the recurrent-network, attention, and transformer labs.

You should write up your results, answers, and findings succinctly, using *no more than two sides of A4* for this investigation. Code need not be reproduced in full: include only short snippets needed to explain your method. This exercise is worth 10 marks (10% of the module).

1 A recurrent sequence-to-sequence model

Sequence-to-Sequence modelling allows one (potentially variable length) input sequence to be translated to another output sequence (potentially of a different length) by transforming the input sequence into a fixed length vector, and using this to drive the generation of the output sequence. There are lots of algorithms and models that can achieve this, but a classic approach is the model proposed by Sutskever et al (`https://arxiv.org/abs/1409.3215`), which uses LSTMs to encode the input to a pair of vectors (the final hidden and cell states), and LSTMs to decode the pair of vectors into the desired output. Sequence-to-sequence models gained popularity as (then) state-of-the-art methods for language translation.

We've implemented a fairly basic sequence-to-sequence model for a translation task and created a dataset from which to train which has the following form:

```
^ .--. .-. . ..-. .- -.-. . / $|^preface $
^ . / ... ..- .--. .--. --- ... $|^e suppos$
```

Each line consists of an input sequence left of the `|` character, and output sequence on the right. The `^` characters represent SOS (Start Of Sequence) tokens, and $ represents EOS (End Of Sequence).

The code and data is available to you in a colab notebook here: `https://tinyurl.com/idl-seq2seq`. The notebook defines an LSTM encoder-decoder model. Most of the implementation is complete, but the `forward` method of the `Encoder` has been omitted. The encoder should embed the input, pass it through its LSTM, and return the final hidden and cell states. Its sequence of LSTM outputs is not used by this model.

1.1 Complete and train the recurrent model (3 marks)

Complete the encoder and train the model. Adapt the training loop to record the losses, include the short code snippet you added, plot the training and validation loss curves, and report the final losses. Briefly explain the shapes and roles of the hidden and cell states passed from encoder to decoder.

1.2 Evaluate and use the model (2 marks)

Evaluate the trained model on the supplied held-out set using both token accuracy and exact-sequence accuracy. Token accuracy should exclude padded positions; exact-sequence accuracy should count a sequence as correct only when every non-padding token, including the end-of-sequence token, is correct. Explain why these measures can give different impressions of performance.

Then use the model to `.- -. ... .-- . .-. / - .... . / ..-. --- .-.. .-.. --- .-- .. -. --.`:

- `.-- .... -.-- / .. ... / - .... . / --- .-. -.. . .-. / --- ..-. / - .... . / --- ..- - .--. ..- - / .-. . ...- . .-. ... . -..`
- `.-- .... .- - / .. ... / - .... . / .--. --- .. -. - / --- ..-. / - . .- -.-. .... . .-. / ..-. --- .-. -.-. .. -. --.`

2 Generalisation beyond the training lengths

The examples used for training contain only a limited range of sequence lengths. A model can perform well on a random held-out subset of these examples without learning an algorithm that generalises to substantially longer sequences.

For this investigation, input length means the number of unpadded input tokens seen by the model, excluding SOS and EOS. Construct longer examples by concatenating two or more existing training examples before numericalisation: remove the outer SOS and EOS tokens, trim the surrounding separator spaces from each Morse-code fragment, join those fragments with a single space, concatenate the corresponding plaintext fragments in the same order, and then restore one SOS token and one EOS token around each complete sequence. This creates unfamiliar sequence lengths from fragments the model encountered during training.

2.1 Test length generalisation (3 marks)

Construct validation sets grouped by input length, including concatenated examples at least two lengths beyond those used during training. Plot token accuracy and exact-sequence accuracy against input length. Include representative successful and unsuccessful decoded sequences.

Explain the behaviour you observe. Your answer should distinguish exposure to particular text fragments from exposure to particular sequence lengths, and should discuss how errors made early by an autoregressive decoder can affect later outputs.

3 Attention and padding masks

The attention lab notebook (`https://tinyurl.com/idl-TransformerLM`) introduced padding masks so that padded positions do not participate as if they were real tokens. To disable only padding masking, temporarily modify `get_attn_pad_mask` so that it returns an all-false Boolean tensor of the same shape as the usual padding mask. Do not bypass `scores.masked_fill_(...)`, because the same operation also applies the decoder's causal mask. Do not otherwise change or retrain the model.

3.1 Investigate the effect of masking (2 marks)

Using the same trained model parameters, evaluate the supplied attention-based model with the correct padding mask and with padding masking disabled as described above. Use batches containing substantially different sequence lengths. Report an appropriate quantitative comparison and one concrete example where padding changes an output or intermediate representation.

Explain why the effect depends on the amount of padding in a batch, and why an apparently small implementation detail can create a systematic modelling error.